Employee Management Dashboard

React.js Single Page Application - Complete Project Documentation

JSX, Components, Props, State, Hooks, React Router, Forms and Context API

Internship	Sumerix Global - Full Stack Web Development Internship
Task	Task 3 of 6 - React.js Fundamentals and Single Page Application Development
Prepared by	Anjan Basak
Intern ID / Roll No.	_____
Mentor / Supervisor	_____
Submission date	07/11/2026
GitHub repository	https://github.com/123anjan/SumerixGlobal-projects-Task-3.git
Live demo (optional)	https://123anjan.github.io/SumerixGlobal-projects-Task-3/

Table of Contents

1. Introduction 3
2. Objectives and Learning Outcomes 3
3. Requirements Analysis 4
4. Tools and Technologies 5
5. System Design 6
6. Data Design 10
7. Implementation 11
8. React Concepts Demonstrated 14
9. Context API and State Management 17
10. React Router and Single Page Application 17
11. UI/UX, Responsiveness and Accessibility 18
12. Screenshots Gallery 19
13. Testing 26
14. Errors Faced and Fixes 27
15. Performance and Best Practices 28
16. Project Plan 28
17. Deployment and GitHub 29
18. Connection to Task 2 29
19. Evaluation Checklist 30
20. Challenges and Lessons Learned 30
21. Future Enhancements and Task 4 31
22. Conclusion 31
23. References 31

1. Introduction

The Employee Management Dashboard is a responsive, accessible single-page application (SPA) built with **React.js** and **Vite**. An administrator logs in, sees a dashboard with summary statistics, browses employees as cards, searches and filters them, opens a details page for each person, and adds, edits or deletes employees using validated forms. The interface supports light and dark mode, works on phones, tablets and desktops, and navigates between pages without reloading the browser.

The project was built as **Task 3** of the Sumerix Global Full Stack Web Development Internship. It extends Task 2 (Interactive Employee Directory): the API, service and storage logic of Task 2 was reused, while the vanilla JavaScript interface was rebuilt as React components with React Router and the Context API. Where Task 2 called a *render()* function by hand, React now redraws the interface automatically whenever state changes.

1.1 Project at a glance

Item	Details
Project title	Employee Management Dashboard
Type	Front-end single-page application (no custom backend)
Language	JavaScript (ES6+), JSX, HTML5, CSS (global CSS and CSS Modules)
Library / build tool	React 19 with Vite and npm
Routing	React Router (react-router-dom): nested, dynamic and protected routes, 404 page
Global state	Context API: ThemeContext, AuthContext, EmployeesContext
Data source	RandomUser public REST API (24 employees on the first load)
Storage	Local Storage with the prefix `ed3:` (employees, theme, auth)
Demo login	`admin` / `admin123` (frontend-only placeholder)
Key features	Login, dashboard, employee list, search, filters, details, add, edit, delete, responsive navigation, dark mode, loading / empty / error states
Deployment	GitHub Pages with the gh-pages package (Vercel and Netlify also possible)

1.2 Why this project matters

Most modern front-end jobs use a component library such as React. Building a complete dashboard teaches how to break a user interface into reusable components, pass data with props, manage state, run side effects, build forms, route between pages and share data globally. These are the skills needed for Task 4 (advanced React, state management and authentication) and for real production applications.

1.3 Scope

- **In scope:** React front-end, routing, forms with validation, Context API, API integration, Local Storage, responsive and accessible UI, documentation and GitHub deployment.
- **Out of scope:** real back-end, database and secure authentication. The login is a frontend-only placeholder because the credentials are visible in the source code; real authentication is planned for Task 4.

2. Objectives and Learning Outcomes

2.1 Objectives

- Understand what React is and why Single Page Applications are used.
- Create a React project with Vite and organise it with a professional folder structure.
- Write JSX using expressions, conditional rendering, lists with keys and fragments.
- Build functional, reusable and composed components and pass data with props.
- Manage data with `useState` and handle side effects with `useEffect`.
- Build controlled forms with validation, submission and error messages.
- Create multi-page navigation with React Router, including dynamic and protected routes.
- Share global data (theme, authentication, employees) with the Context API.
- Use `useRef`, `useMemo`, `useCallback` and custom hooks.
- Style the application with CSS Modules and global CSS, and make it responsive and accessible.
- Handle loading, empty and error states and publish the project on GitHub.

2.2 Learning outcomes

After completing this task the learner can build a multi-page React application, split a design into components, pass props and lift state, validate forms, protect routes, use Context and custom hooks, persist data in the browser, test the application on several devices, and deploy a Vite project to GitHub Pages.

3. Requirements Analysis

3.1 Functional requirements

ID	Requirement	Implemented in
F1	Login page with demo credentials; logged-out users cannot open protected pages	LoginPage, AuthContext, ProtectedRoute
F2	Dashboard with totals, availability, department count, department bars and recent employees	DashboardPage, StatCard
F3	Employee list shown as cards with photo or initials	EmployeesPage, EmployeeCard, Avatar
F4	Search by name (debounced)	EmployeeFilters, useDebounce
F5	Filter by department and availability, plus Clear filters	EmployeeFilters, EmployeesPage
F6	Employee details page for each employee	EmployeeDetailsPage (/employees/:id)
F7	Add employee with validation	AddEmployeePage, EmployeeForm
F8	Edit employee with a pre-filled form	EditEmployeePage, EmployeeForm
F9	Delete employee after confirmation	EmployeeCard / details page, EmployeesContext
F10	Responsive navigation (hamburger menu on mobile)	Navbar, Navbar.module.css
F11	Dark mode that follows the system and is remembered	ThemeContext, global.css
F12	Loading, empty and error states with Retry	Spinner, EmptyState, ErrorMessage
F13	Data kept after refresh; reset to fresh data	EmployeesContext, storage.js
F14	404 page for unknown URLs	NotFoundPage, AppRoutes

3.2 Non-functional requirements

- **Usability:** clear layout, visible feedback, consistent buttons and forms.
- **Responsiveness:** works from about 360 px (phone) to wide desktop screens.
- **Accessibility:** labels, keyboard support, ARIA attributes, focus outlines and readable contrast.
- **Maintainability:** small single-purpose files, reusable components and clear naming.
- **Performance:** data cached in Local Storage, debounced search, `memo`, `useMemo`, `useCallback`.
- **Reliability:** safe storage wrapper, error screens with Retry, safe storage access, no crashes on bad data.
- **Portability:** runs on current Chrome, Firefox and Edge, and deploys to GitHub Pages.

3.3 Mapping to the Task 3 brief

Brief part	Topic	Where it appears
A, B	SPA concept, Vite with React, folder structure, npm scripts	Sections 4, 5, 7
C	JSX: expressions, conditional rendering, lists, keys, fragments	Sections 7, 8
D, E	Components and props	Sections 5, 7, 8
F	State, events, useEffect, dependency arrays	Sections 7, 8
G	Forms: controlled inputs, validation, submission	Section 7
H	React Router: dynamic, nested and protected routes	Section 10
I	Context API: theme and authentication contexts	Section 9
J	Hooks: useRef, useMemo, useCallback, custom hooks	Sections 7, 8
K	Styling: CSS Modules, global CSS, responsive design, icons, fonts	Section 11
L	Loading, empty and error states, fallback UI	Sections 7, 11
M, N	Mini project: Employee Management Dashboard	Whole document

4. Tools and Technologies

Technology	Purpose
React 19 (JSX)	Builds the interface from components
Vite	Fast development server, hot reload and optimised production build
react-router-dom	Client-side routing for the single-page application
Context API	Global state for theme, authentication and employees
CSS Modules and global CSS	Component-scoped styles, CSS variables and dark mode
react-icons (Feather)	Icons
Google Fonts (Inter)	Typography
Fetch API	Reads employee data from the REST API
RandomUser API	Source of the employee data (name, email, phone, photo, city)
ESLint	Finds code-quality problems
gh-pages	Publishes the build to GitHub Pages
Git and GitHub	Version control and code hosting
React Developer Tools	Inspect the component tree, props and state
Chrome / Firefox DevTools	Debugging, network throttling, device toolbar, Lighthouse

4.1 Prerequisites

Tool	Check command
Node.js LTS	node -v
npm	npm -v
Git	git --version

5. System Design

5.1 Layered architecture

The application is divided into layers. Pages and components never call the API directly: they ask a context, which uses a service, which uses the API or storage. This keeps every file focused on one job and makes the project easy to grow in Task 4.

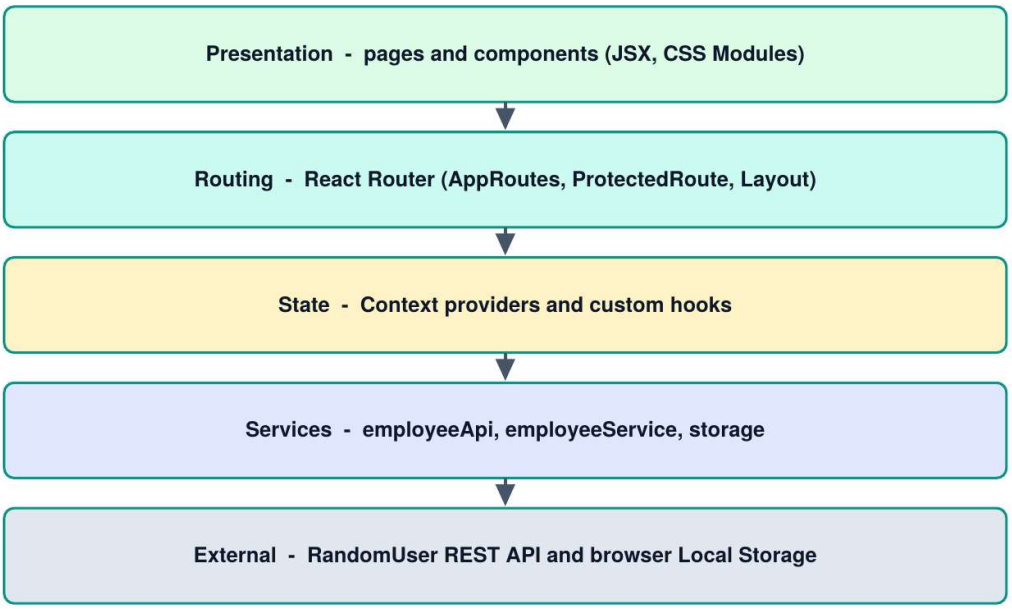


Figure 1: Layered architecture of the Employee Management Dashboard

5.2 Component tree

The providers wrap the whole application so that every page can use theme, authentication and employee data. **ProtectedRoute** guards all pages except login, and **Layout** supplies the navigation bar and an *Outlet* where the current page is drawn.

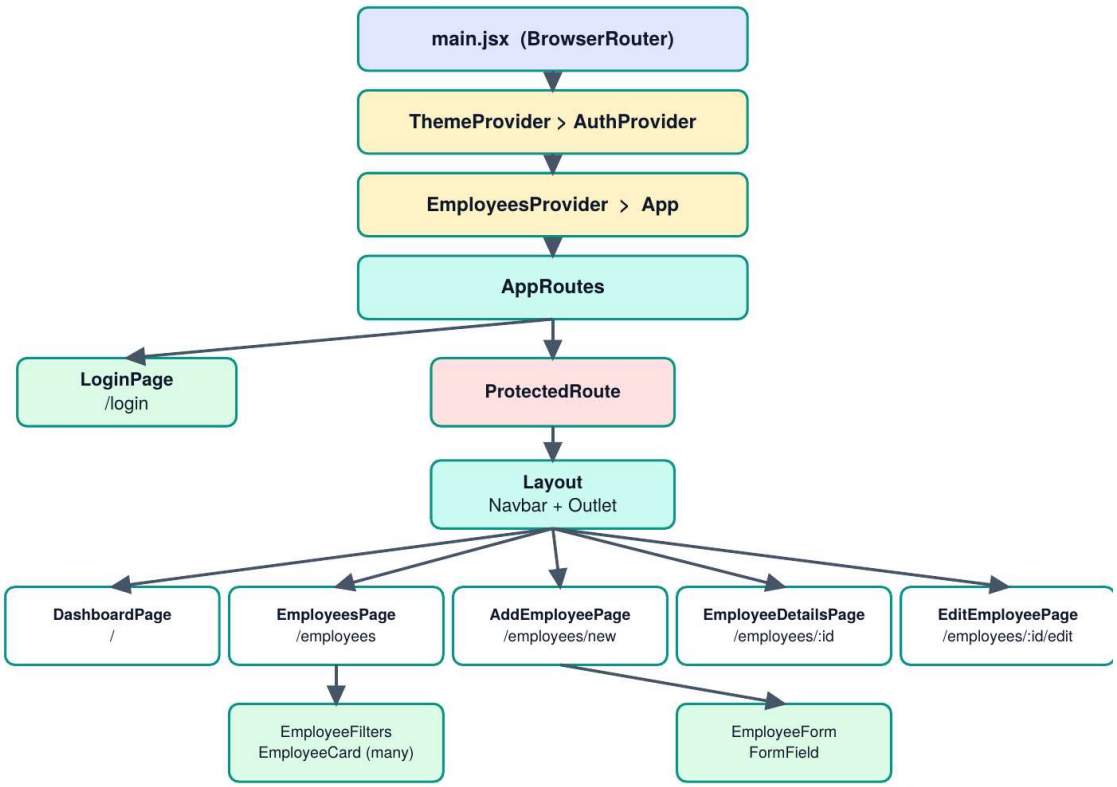


Figure 2: Component tree (providers, routes, layout, pages and child components)

5.3 Folder structure

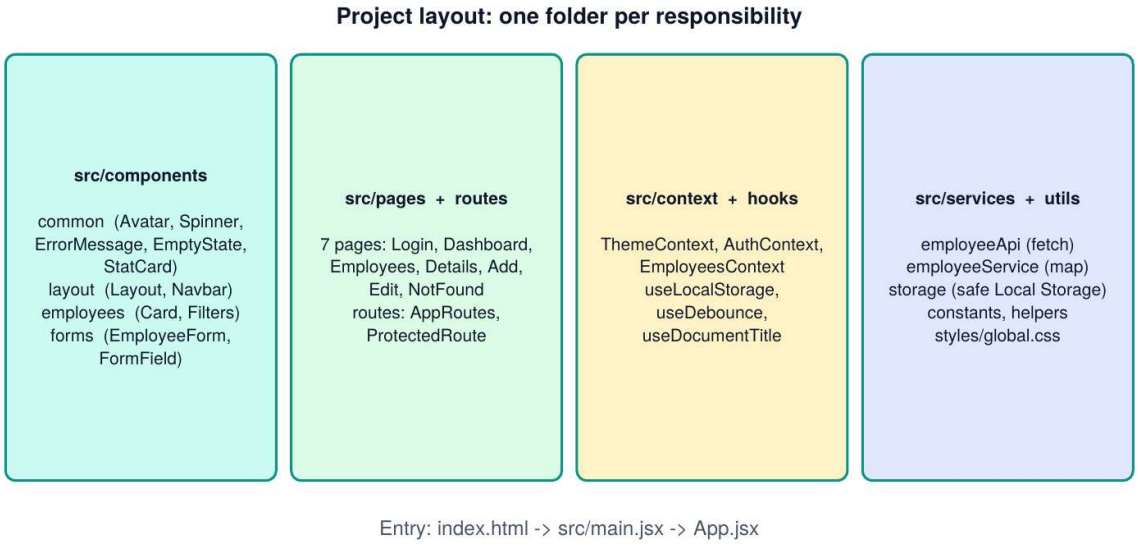


Figure 3: Folder responsibilities

```
Task-3/
|-- public/                    favicon.svg
|-- src/
|   |-- components/
|   |   |-- common/     Avatar, EmptyState, ErrorMessage, Spinner, StatCard
|   |   |-- layout/     Layout, Navbar (+ Navbar.module.css)
|   |   |-- employees/  EmployeeCard (+ module.css), EmployeeFilters
|   |   `-- forms/      EmployeeForm, FormField
|   |-- context/        ThemeContext, AuthContext, EmployeesContext
|   |-- hooks/          useLocalStorage, useDebounce, useDocumentTitle
|   |-- pages/          Login, Dashboard, Employees, EmployeeDetails,
|   |                   AddEmployee, EditEmployee, NotFound
|   |-- routes/         AppRoutes, ProtectedRoute
|   |-- services/       employeeApi, employeeService, storage
|   |-- styles/         global.css
|   |-- utils/          constants.js, helpers.js
|   |-- App.jsx
|   `-- main.jsx
|-- index.html   package.json   vite.config.js   eslint.config.js
`-- README.md    .gitignore
```

Each file has one responsibility (single responsibility principle). Reusable pieces live in `components/common`, feature pieces in their own folders, and logic that is not visual lives in `services`, `hooks` and `utils`.

5.4 Routes

URL	Page	Access	Notes
/login	LoginPage	Public	Logged-in users are redirected away
/	DashboardPage	Protected	Statistics and recent employees
/employees	EmployeesPage	Protected	Search, filters and cards
/employees/new	AddEmployeePage	Protected	Validated form
/employees/:id	EmployeeDetailsPage	Protected	:id is a route parameter
/employees/:id/edit	EditEmployeePage	Protected	Pre-filled form
*** (anything else)**	NotFoundPage	Public	404 page

5.5 Data flow

On the very first visit nothing is stored, so **EmployeesContext** fetches 24 employees from the API, converts them with **toEmployee()** and saves them in Local Storage. On later visits the list is read from storage, so the app opens instantly and keeps the user's changes. Add, edit and delete update the context state, and the new list is saved again automatically.

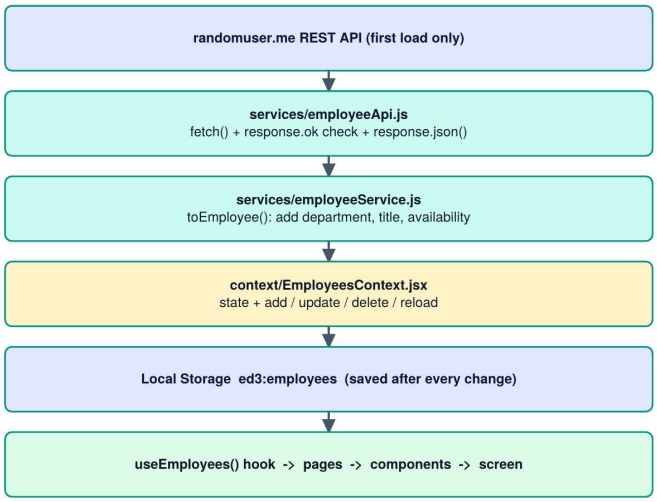


Figure 4: Data flow from the API to the screen

5.6 Wireframes

The wireframes show the main layouts before they were coded. They are simplified; the screenshots in Section 12 show the final result.

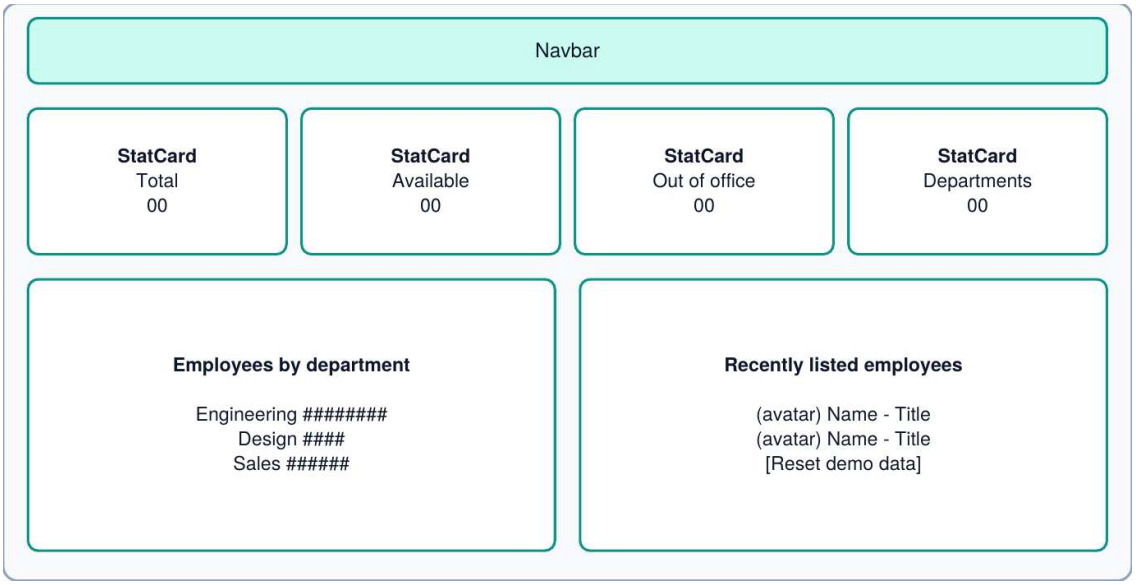


Figure 5: Dashboard wireframe

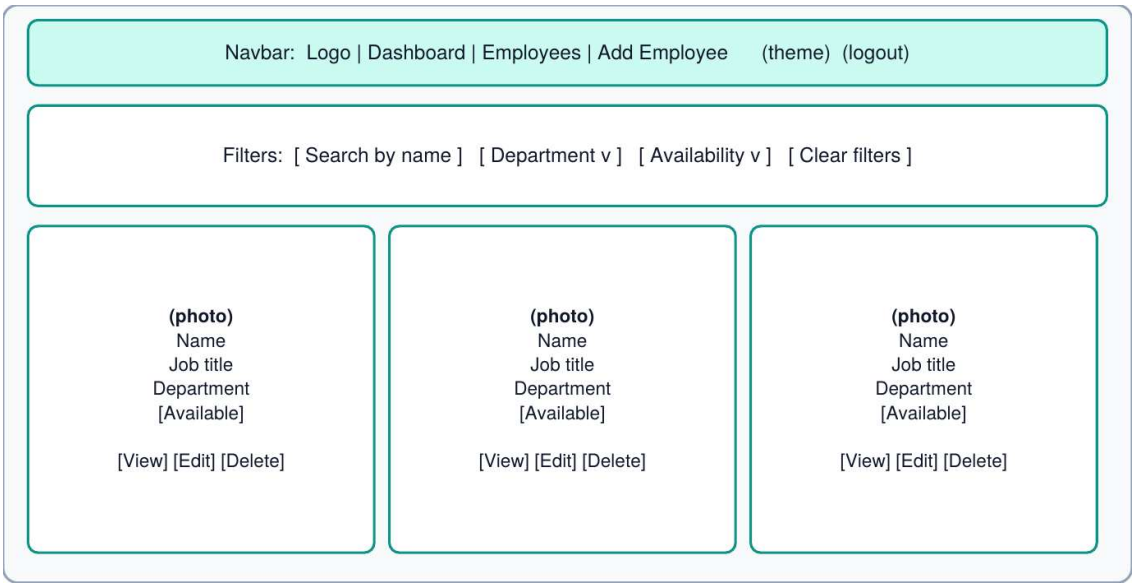


Figure 6: Employee list wireframe

6. Data Design

6.1 API request

```
GET
https://randomuser.me/api/?results=24&seed=sumerix&nat=us,gb,au&inc=login,name,email,phone,picture,locati
on
```

Parameter	Meaning
results=24	Number of users returned
seed=sumerix	Returns the same people every time
nat=us,gb,au	Nationalities included
inc=...	Only the listed fields are returned (smaller response)

RandomUser does not provide departments, job titles or availability, so `toEmployee()` in `employeeService.js` assigns them in a predictable way from constants.

6.2 Employee object

```
{
  id, name, title, department,
  availability,        // 'Available' | 'Out'
  email, phone, city,
  photo                // '' for employees added in the form
}
```

6.3 Browser storage

Key	Contains
ed3:employees	The full employee list, including edits, additions and deletions
ed3:theme	"light" or "dark"
ed3:auth	The logged-in username, or null

The **ed3:** prefix keeps Task 3 data separate from Task 2 (**ed:**), because GitHub Pages sites of the same account share one browser origin. To inspect storage open DevTools, then Application, then Local Storage. To reset everything run `localStorage.clear()` or use **Reset demo data** on the dashboard.

6.4 Application states

State	When it happens	What the user sees
Loading	First load or after Reset demo data	Spinner and message (role=status)
Success	Data is ready	Employee list and result count
Empty	No employee matches search or filters	No employees found and a Clear filters button
Error	The API request fails	Error message and a Retry button (role=alert)

7. Implementation

7.1 Project setup

The project was created with Vite's React template (JavaScript, not TypeScript) and ESLint. Two runtime packages and one development package were added.

```
cd "SumerixGlobal projects"
npm create vite@latest Task-3 -- --template react
cd Task-3
npm install
npm install react-router-dom react-icons
npm install -D gh-pages
npm run dev          # http://localhost:5173
```

vite.config.js sets the base path only for production builds, so the dev server uses `/` while GitHub Pages uses the repository name. **package.json** adds the `postbuild` step that copies `index.html` to `404.html` so that page refreshes on nested URLs work on GitHub Pages.

```
// vite.config.js
export default defineConfig(({ command }) => ({
  plugins: [react()],
  base: command === 'build' ? '/YOUR-REPO-NAME/' : '/',
}));

// package.json (scripts)
"dev": "vite", "build": "vite build",
"postbuild": "node -e \"require('fs').copyFileSync('dist/index.html','dist/404.html')\"",
"preview": "vite preview", "lint": "eslint .",
"predeploy": "npm run build", "deploy": "gh-pages -d dist"
```

7.2 Entry point and providers

main.jsx wraps the application in the router and in the three providers. The router uses `import.meta.env.BASE_URL` as its basename so that links work both locally and on GitHub Pages.

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter basename={import.meta.env.BASE_URL}>
      <ThemeProvider><AuthProvider><EmployeesProvider>
        <App />
      </EmployeesProvider></AuthProvider></ThemeProvider>
    </BrowserRouter>
  </StrictMode>
);
```

7.3 Routing and protected routes

AppRoutes declares the public login route, a protected group rendered inside **Layout**, nested employee routes, the dynamic `/employees/:id` route and a catch-all 404. **ProtectedRoute** reads the user from AuthContext; when nobody is logged in it redirects to `/login` and remembers the requested page so the user returns to it after logging in.

```
export default function ProtectedRoute() {
  const { user } = useAuth();
  const location = useLocation();
  if (!user) return <Navigate to="/login" replace state={{ from: location }} />;
  return <Outlet />;
}
```

7.4 Context providers

Context	Provides	Key behaviour
ThemeContext	theme, toggleTheme	Starts from the system preference, saves the choice with useLocalStorage and sets data-theme on the html element
AuthContext	user, login(), logout()	Checks the demo credentials, saves the username in ed3:auth
EmployeesContext	employees, loading, error, addEmployee(), updateEmployee(), deleteEmployee(), reload(), reset()	Loads from storage or API on mount, saves after every change, exposes Retry

Each context has a custom hook (`useTheme`, `useAuth`, `useEmployees`) that throws a clear error if it is used outside its provider.

7.5 Custom hooks

Hook	Purpose
useLocalStorage(key, initial)	Works like useState but saves to Local Storage whenever the value changes
useDebounce(value, delay)	Returns the value only after the user stops typing; clears its timer on cleanup
useDocumentTitle(title)	Sets the browser tab title for each page

```
export function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);          // cleanup on change / unmount
  }, [value, delay]);
  return debounced;
}
```

7.6 Services and utilities

File	Responsibility
services/employeeApi.js	fetch() call; checks response.ok and throws a readable error for HTTP failures
services/employeeService.js	toEmployee(): converts a raw API user into an employee object
services/storage.js	Safe Local Storage wrapper using JSON.parse / JSON.stringify inside try/catch
utils/constants.js	Departments, job titles and storage keys
utils/helpers.js	Small helpers (for example initials and id creation)

7.7 Components

Component	Props / role
Avatar	name, photo: shows the photo or the initials when there is no photo
StatCard	label, value: number card for the dashboard
EmptyState	title, children: nothing-found message that can contain a button
ErrorMessage	message, onRetry: error text and Retry button
Spinner	Loading indicator with role=status; slows down for reduced-motion users
Layout / Navbar	Page frame with a skip link, responsive menu, theme and logout buttons, and the Outlet
EmployeeCard (memo)	employee, onDelete: one card with View, Edit and Delete actions; memo avoids needless re-renders
EmployeeFilters	Controlled search box and selects; focuses the search field on mount with useRef
EmployeeForm / FormField	Shared add / edit form with validation; FormField wraps label, input and error text

7.8 Employees page (state, derived data and handlers)

EmployeesPage keeps the search text, department and availability in `useState`. The search text is debounced, the visible list is derived with `useMemo`, and the delete handler is wrapped in `useCallback` so the memoised cards receive a stable function.

```
const [query, setQuery] = useState('');
const [department, setDepartment] = useState('All');
const [availability, setAvailability] = useState('All');
const debouncedQuery = useDebounce(query, 300);
```

```
const visible = useMemo(() => employees.filter(e =>
  e.name.toLowerCase().includes(debouncedQuery.trim().toLowerCase()) &&
  (department === 'All' || e.department === department) &&
  (availability === 'All' || e.availability === availability)
), [employees, debouncedQuery, department, availability]);

const handleDelete = useCallback((id) => {
  if (window.confirm('Delete this employee?')) deleteEmployee(id);
}, [deleteEmployee]);
```

7.9 Forms and validation

EmployeeForm is a controlled component: every input's value comes from state and every change updates state. On submit the default browser behaviour is prevented, the values are validated, and errors are shown next to the fields with `role="alert"` and `aria-describedby`. Valid data is passed to the parent, which calls the context and then navigates to the list with `useNavigate()`.

Field	Rule
Name	Required, at least 2 characters
Email	Required, valid email format
Phone	Optional, digits and common phone symbols only
Job title / Department	Required
Availability	Available or Out
City	Optional

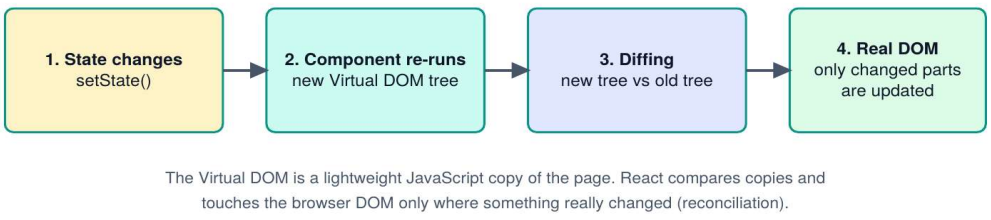
7.10 Styling

- **Global CSS** (`styles/global.css`): colour variables, light and dark themes through `[data-theme='dark']`, buttons, forms, grids and shared classes.
- **CSS Modules** (`Navbar.module.css`, `EmployeeCard.module.css`): styles scoped to one component, so class names never clash.
- **Icons:** react-icons (Feather). **Font:** Inter from Google Fonts, loaded in index.html.

8. React Concepts Demonstrated

8.1 Virtual DOM and rendering

React keeps a lightweight copy of the page in memory (the Virtual DOM). When state changes, React runs the component again, compares the new tree with the old one (diffing) and updates only the parts of the real DOM that changed. This is why typing in the search box updates the list without any manual DOM code.



The Virtual DOM is a lightweight JavaScript copy of the page. React compares copies and touches the browser DOM only where something really changed (reconciliation).

Figure 7: How React updates the screen

8.2 JSX

- **Expressions** in curly braces: `{employee.name}`.
- **Conditional rendering:** `{loading ? : }` and `{error && }`.
- **Lists with keys:** `visible.map(e => )`.
- **Fragments** return several elements without an extra wrapper.
- JSX uses `className` and `htmlFor` instead of `class` and `for`.

8.3 Components and props

Components are functions that return JSX. They are *reusable* (EmployeeCard is drawn many times), *composed* (EmployeeCard contains Avatar) and configured with **props**. Props are read-only and flow from parent to child; a child talks to its parent by calling a function received as a prop. The special `children` prop lets EmptyState wrap a button, and default values keep components safe when a prop is missing.

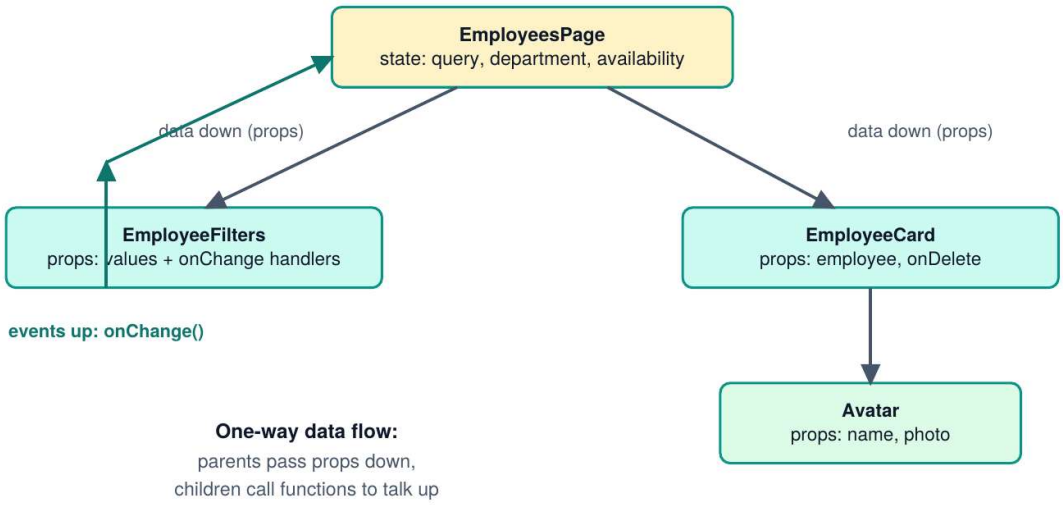


Figure 8: Props flow down, events flow up

8.4 State and events

State is data that changes over time and that the screen depends on. `useState` returns the value and a setter; calling the setter schedules a re-render. Values that can be calculated from existing state (the filtered list, the dashboard counts, the `loading` flag in the context) are **derived** instead of stored, which avoids out-of-sync data.

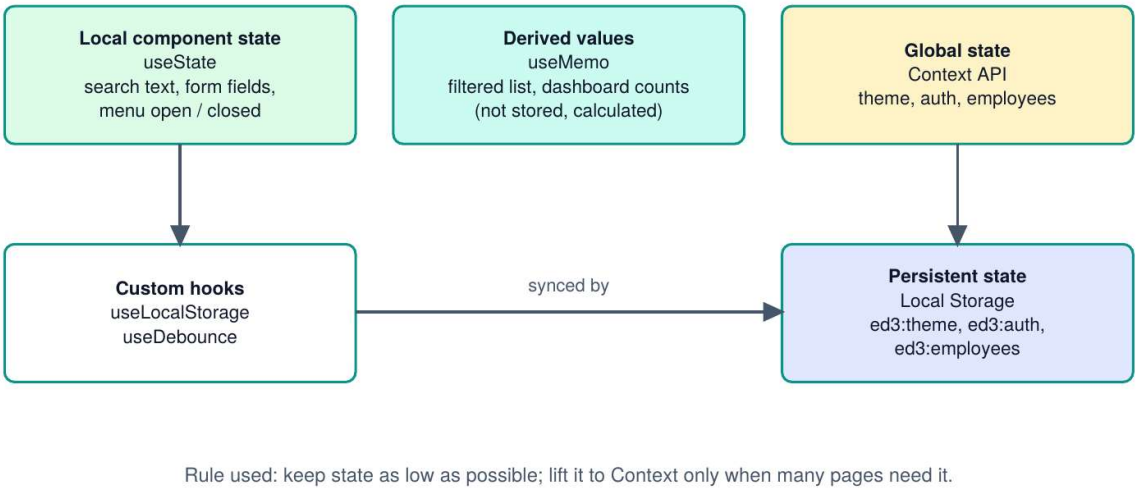


Figure 9: Kinds of state used in the project

8.5 Component lifecycle with useEffect

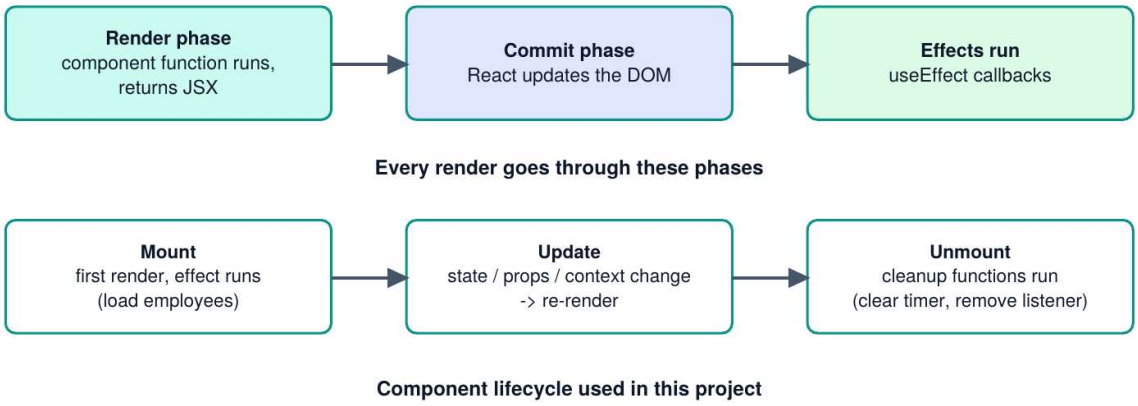


Figure 10: Render, commit and effect phases; mount, update and unmount

Lifecycle idea	Where it is used
Mounting (runs once when the component appears)	EmployeesContext loads data; EmployeeFilters and EmployeeForm focus their first field with useRef
Updating (runs when a value changes)	useLocalStorage saves when its value changes; useDocumentTitle updates the tab title
Unmounting / cleanup	useDebounce clears its timer; Navbar removes its keydown listener

8.6 Hooks summary

Hook	Used for	Example in the project
useState	Local state	Search text, form values, menu open
useEffect	Side effects	Loading employees, saving to storage, key listener
useRef	Reference to a DOM element	Auto-focus the search box
useMemo	Cache a calculated value	Filtered list, dashboard statistics
useCallback	Cache a function	Delete handler passed to memoised cards
useContext	Read global data	useTheme, useAuth, useEmployees
Custom hooks	Reusable logic	useLocalStorage, useDebounce, useDocumentTitle
Router hooks	Navigation	useNavigate, useParams, useLocation

8.7 Rules of hooks

- Call hooks only at the top level of a component or custom hook (not inside loops, conditions or nested functions).
- Call hooks only from React functions.
- List every value used inside `useEffect`, `useMemo` and `useCallback` in the dependency array.
- Never change state directly; always use the setter and create new arrays or objects.

9. Context API and State Management

Without Context, theme, user and employee data would have to be passed through many layers of components as props (prop drilling). A Context holds the value once; a Provider makes it available to everything below it; and a component reads it with `useContext` (wrapped in a custom hook).

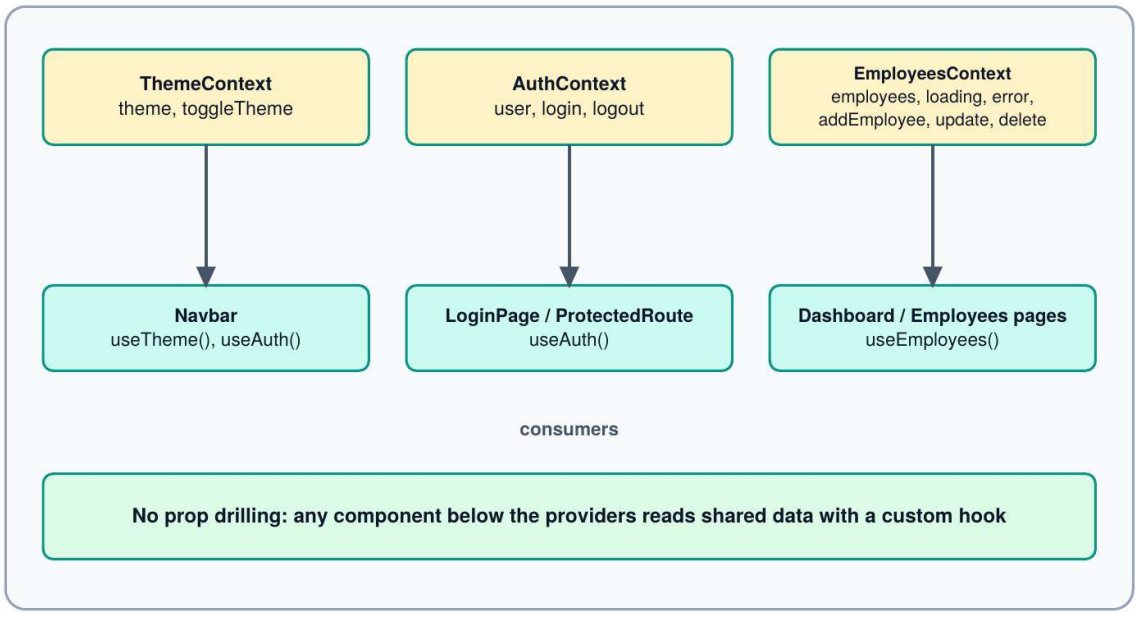


Figure 11: Context architecture: three providers and their consumers

Context	Consumed by
ThemeContext	Navbar (toggle button, aria-pressed)
AuthContext	LoginPage, ProtectedRoute, Navbar (logout)
EmployeesContext	DashboardPage, EmployeesPage, EmployeeDetailsPage, AddEmployeePage, EditEmployeePage

Design decision: local UI state (search text, menu open) stays inside the component that owns it. Only data needed by many pages is placed in Context, to avoid unnecessary re-renders of the whole tree.

10. React Router and Single Page Application

In a traditional website every click requests a new HTML page from the server. In a single-page application the browser loads one HTML page once, and React Router swaps the visible components according to the URL, without a page reload. This makes navigation fast and keeps application state in memory.

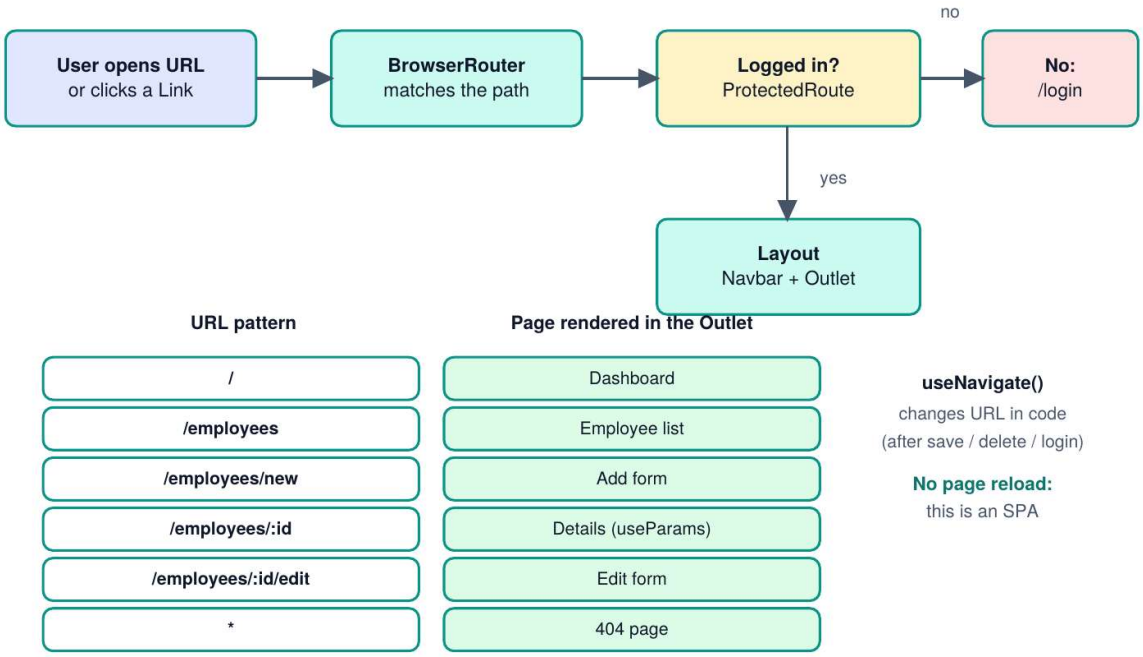


Figure 12: Navigation flow with protected routes

Router feature	Where it is used
BrowserRouter with basename	main.jsx
Routes and Route (nested)	AppRoutes.jsx
Outlet	Layout.jsx and ProtectedRoute.jsx
Link and NavLink (active style)	Navbar, cards, buttons
useNavigate	After login, save, delete and logout
useParams	EmployeeDetailsPage and EditEmployeePage read :id
Navigate and useLocation	ProtectedRoute redirects and remembers the target page
Catch-all route (*)	NotFoundPage

Because GitHub Pages cannot serve nested URLs such as `/employees/3` on refresh, the build copies `index.html` to `404.html`, and the base path is set in `vite.config.js`.

11. UI/UX, Responsiveness and Accessibility

11.1 Responsive design

Screen	Behaviour
Mobile (under 720 px)	Hamburger menu; employee cards in one column
Tablet (about 768 px)	Two columns of cards; filters in a row
Desktop (1280 px and above)	Full navigation bar; wider card grid; content width up to 1100 px

11.2 Dark mode

The theme starts from the user's system setting (`prefers-color-scheme`), then remembers the user's choice in Local Storage. Colours are CSS variables, and the `data-theme` attribute on the html element switches the variable values, so no component needs theme-specific code.

11.3 Accessibility

- Skip-to-content link for keyboard users.
- Every input has a label; errors use `role="alert"` and `aria-describedby`.
- Everything works with the keyboard; Esc closes the mobile menu; visible `:focus-visible` outlines.
- `aria-live="polite"` announces the number of matching employees.
- `aria-expanded` and `aria-controls` on the menu button; `aria-pressed` on the theme button.
- Icon-only buttons have descriptive labels such as "Delete Anna Smith".
- Readable contrast in both themes; the spinner respects reduced-motion settings.
- Semantic HTML: header, nav, main, section, article and definition lists.

12. Screenshots Gallery

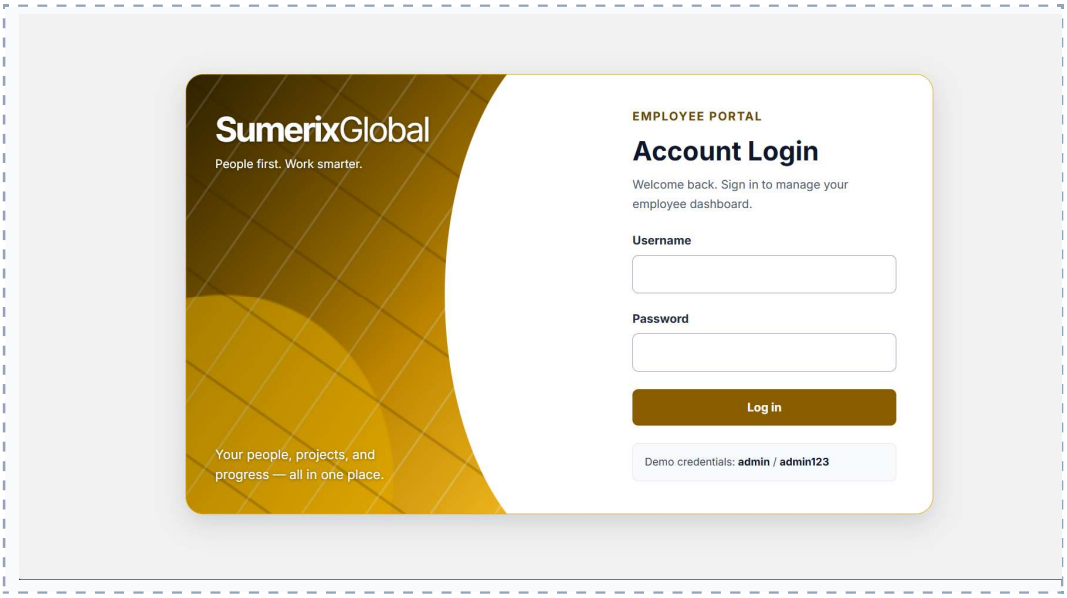


Figure 13: Login page with demo credentials

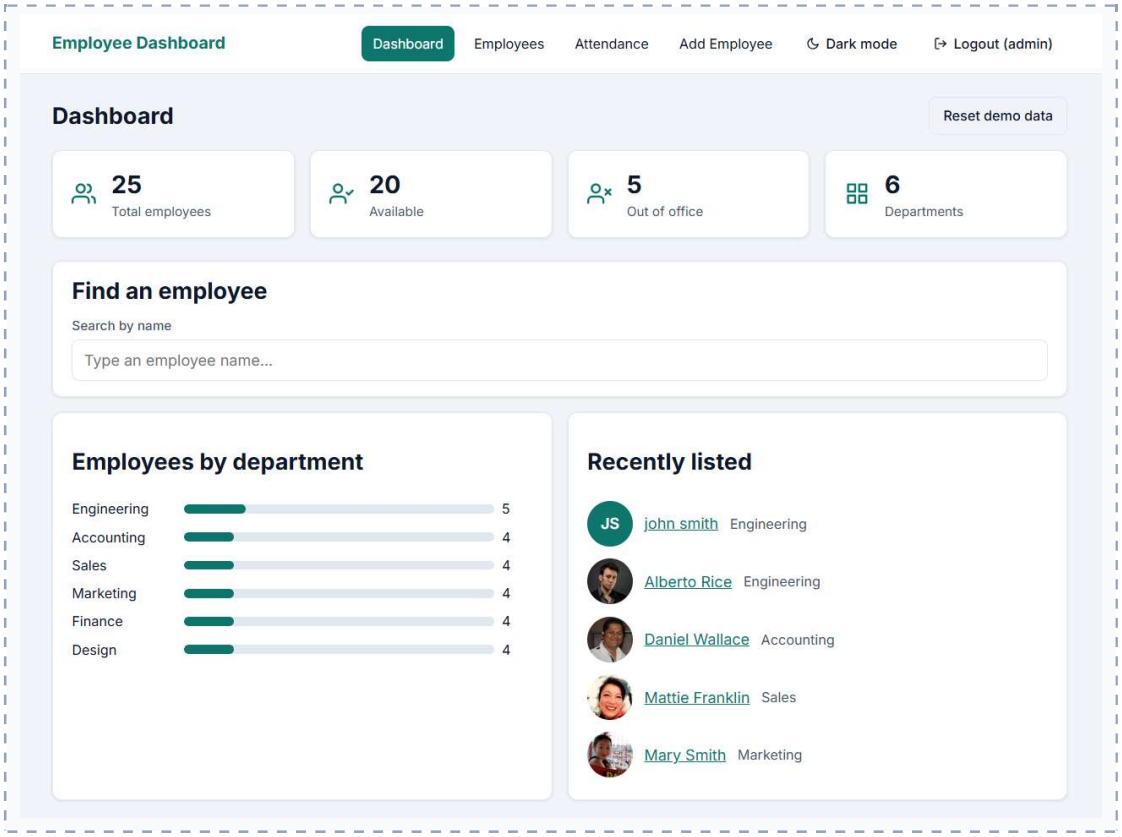


Figure 14: Dashboard with statistic cards and department bars

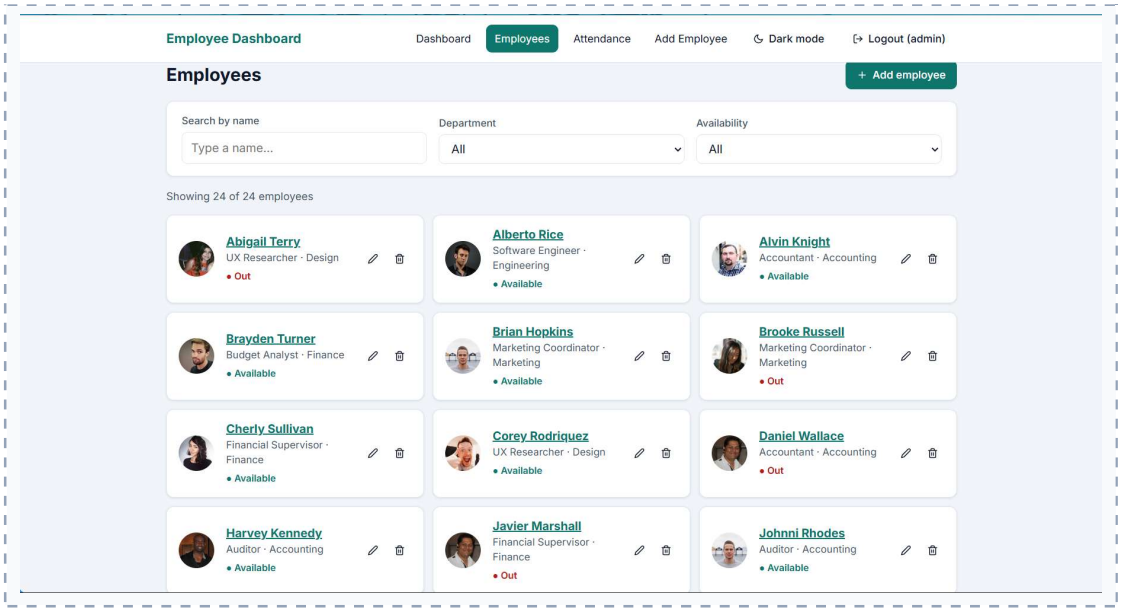


Figure 15: Employee list as cards

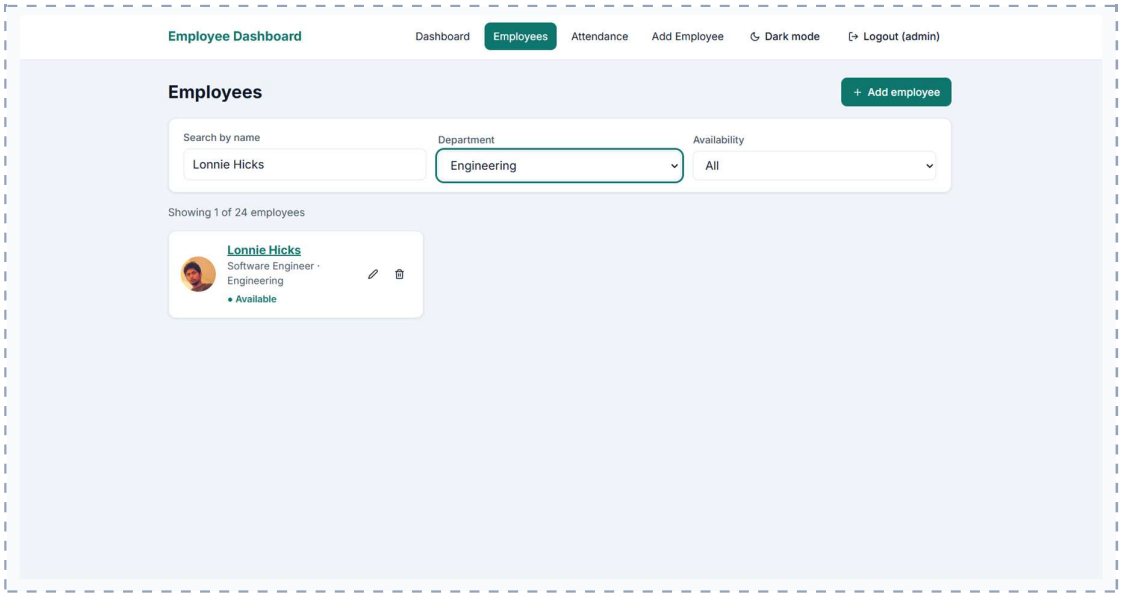


Figure 16: Search and filters in use

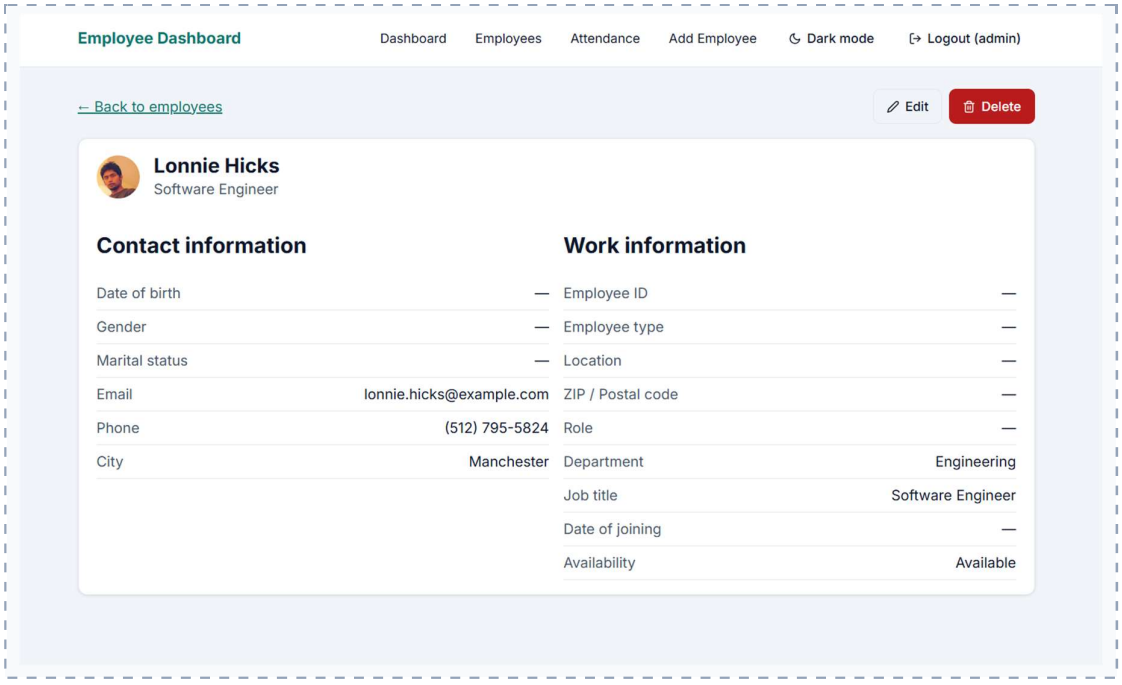


Figure 17: Employee details page

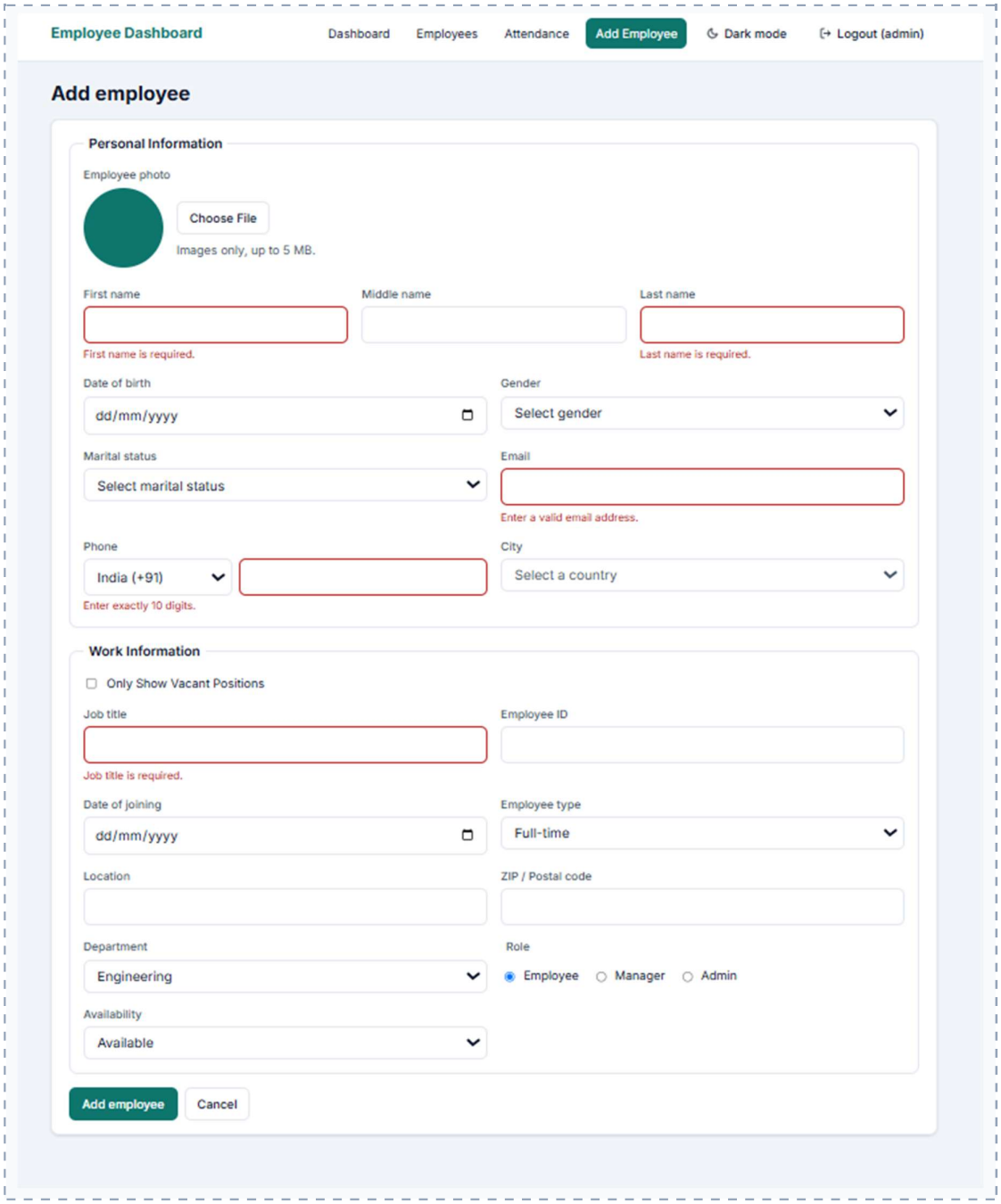


Figure 18: Add employee form showing validation errors

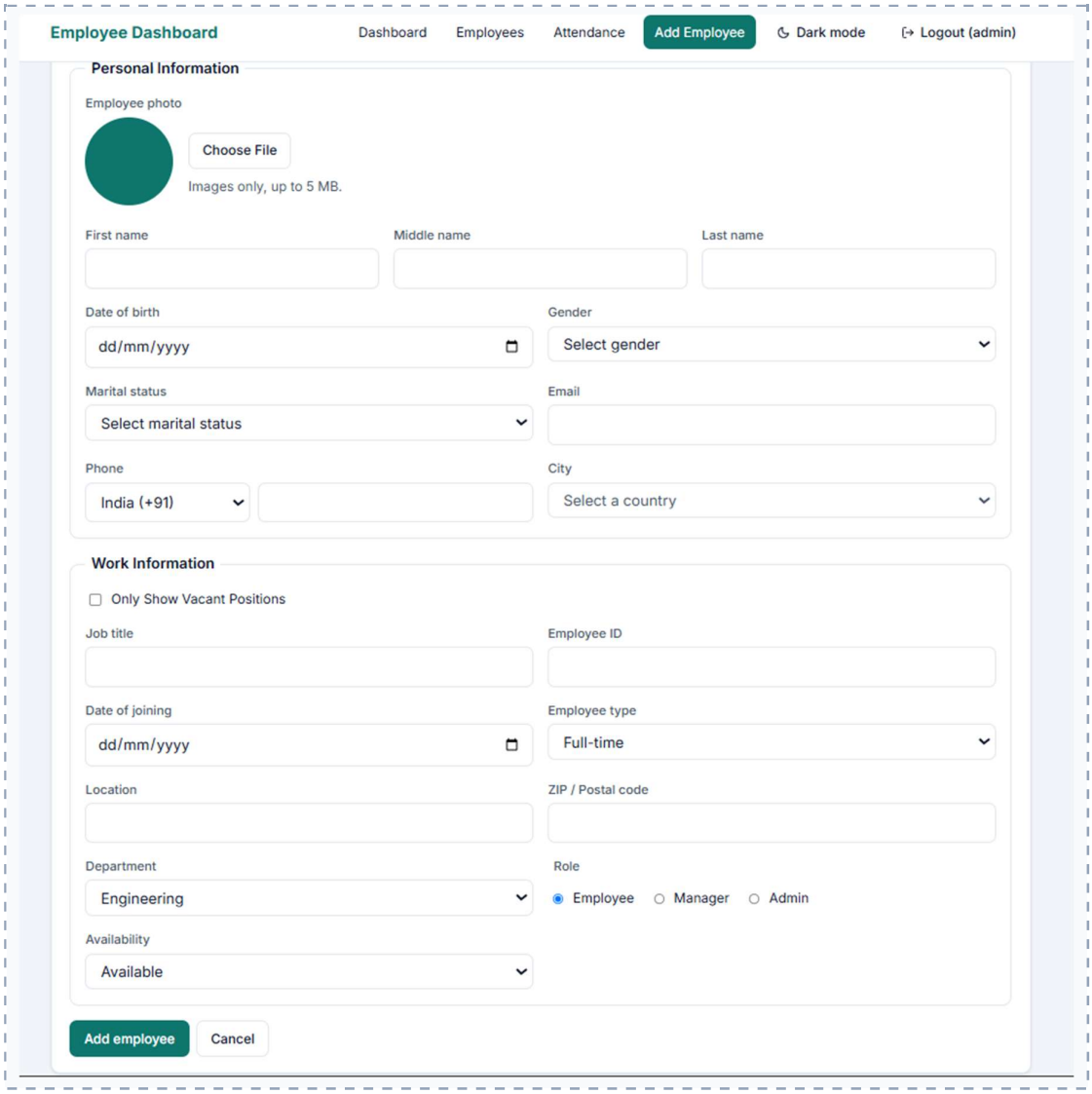


Figure 19: Edit employee form (pre-filled)

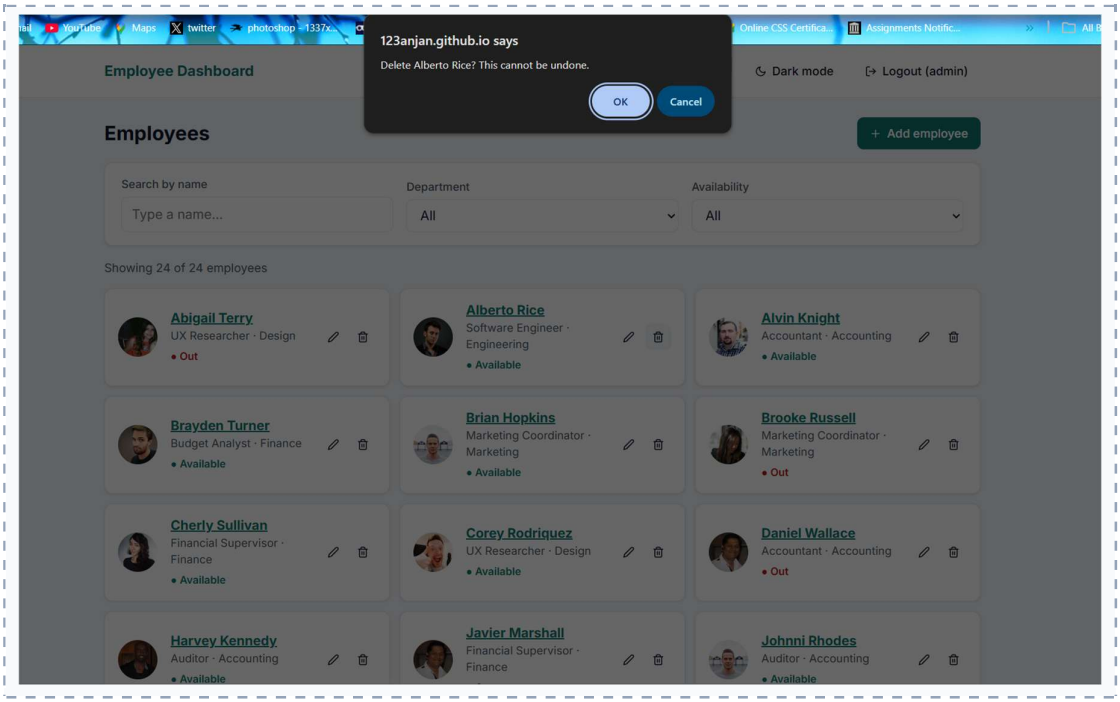


Figure 20: Delete confirmation prompt

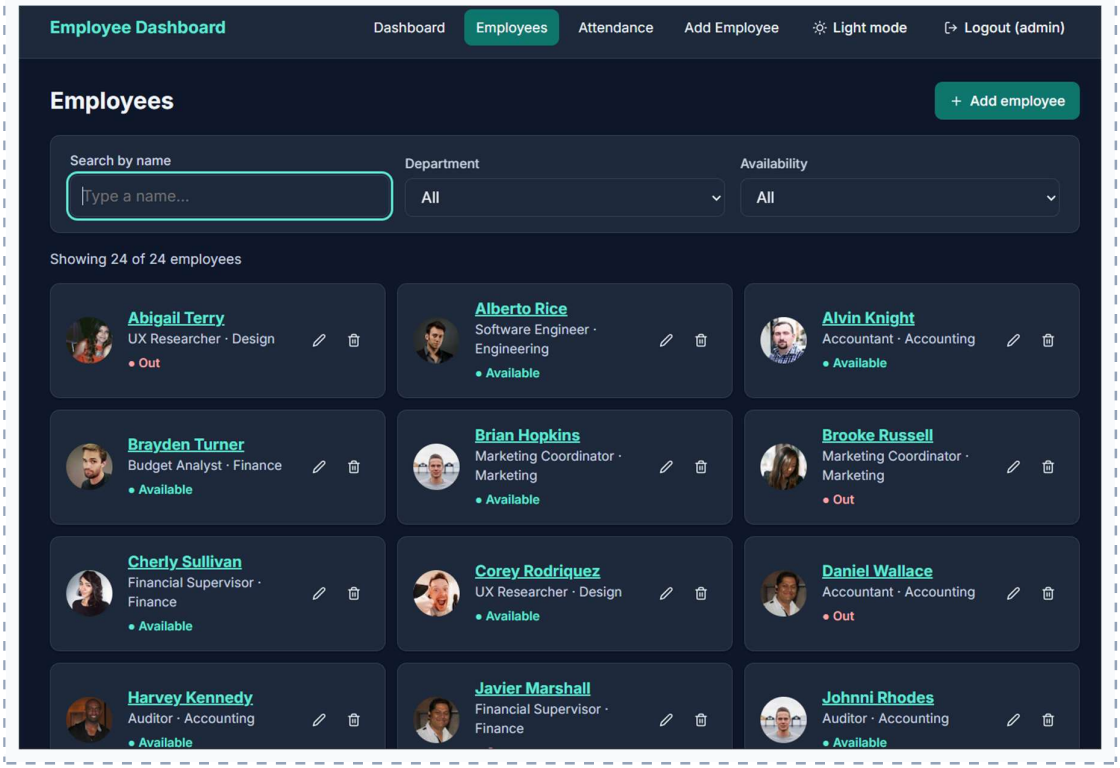


Figure 21: Dark mode

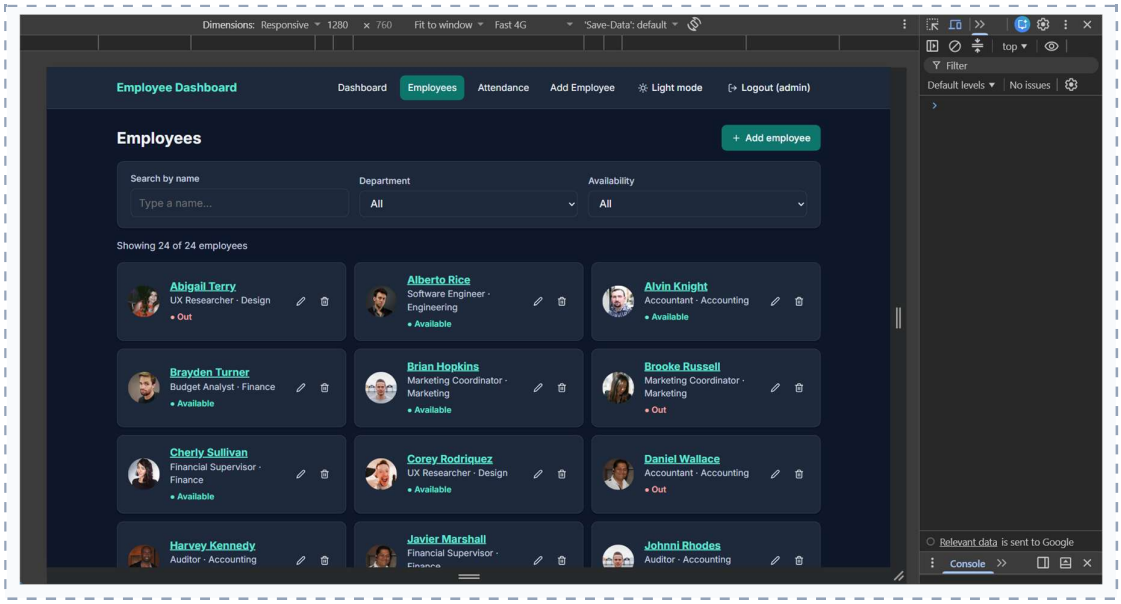


Figure 22: Desktop layout

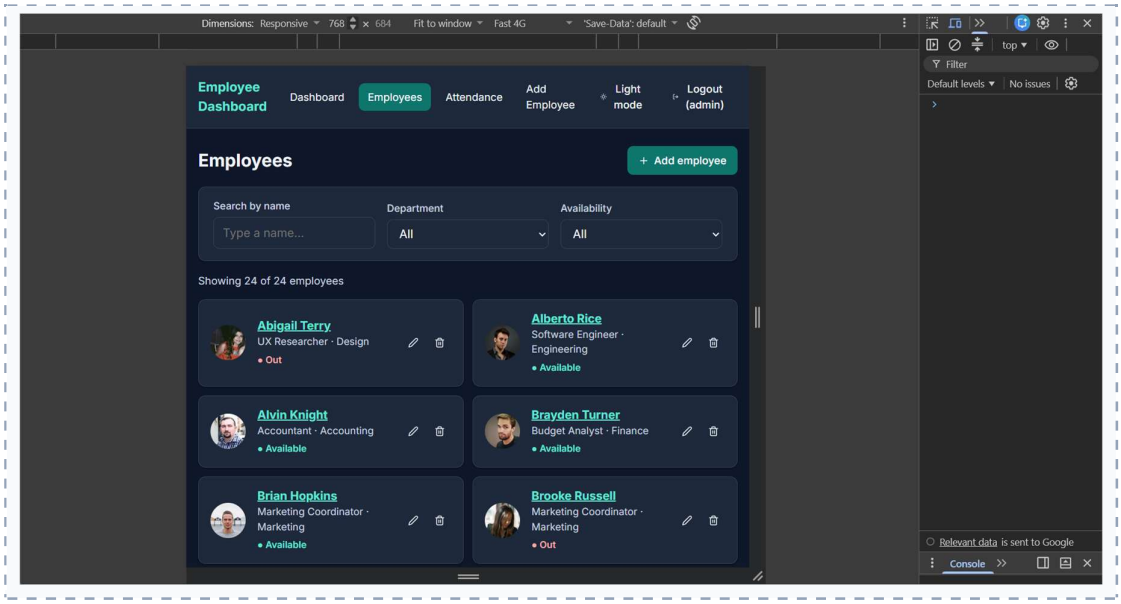


Figure 23: Tablet layout

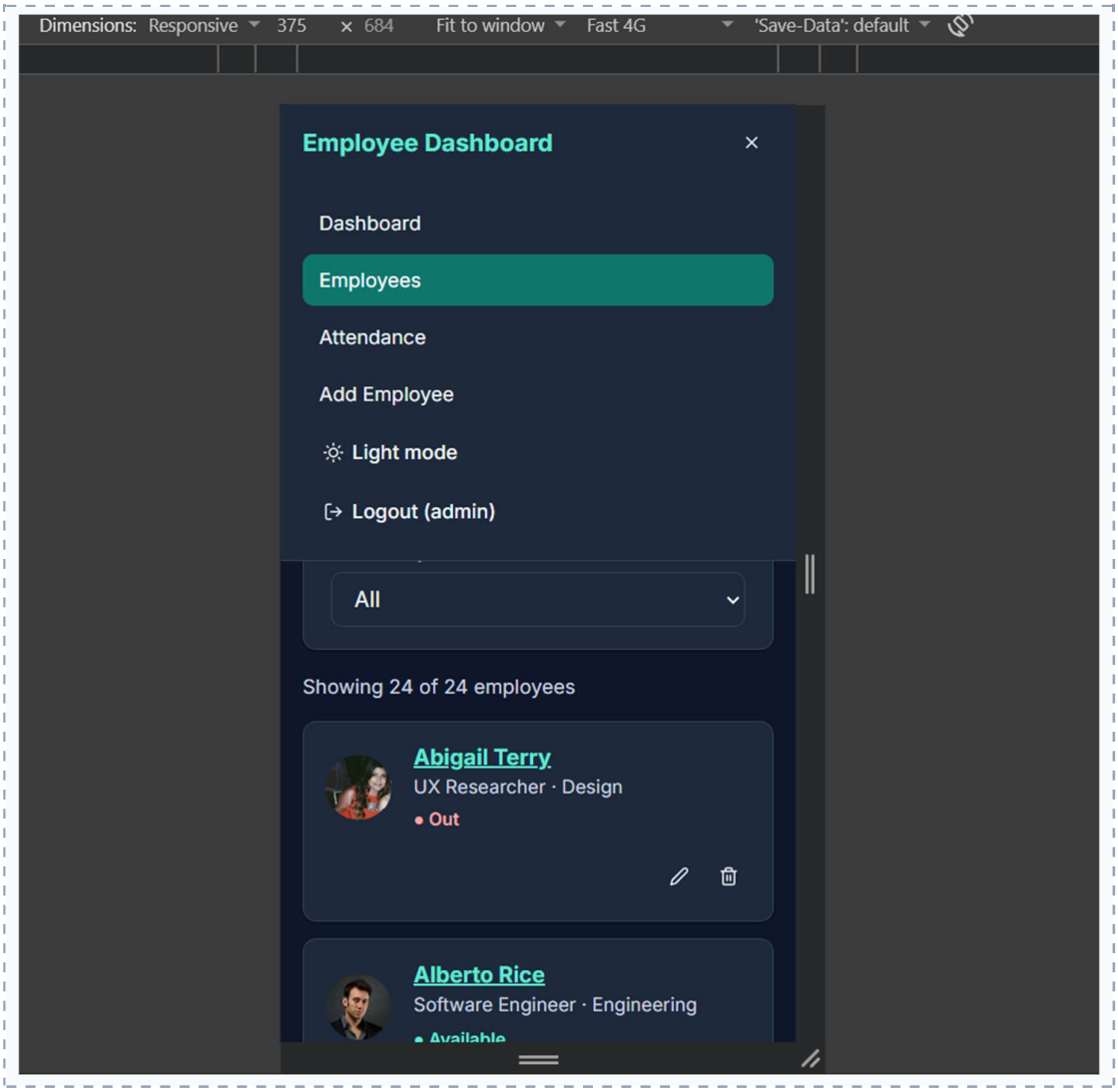


Figure 24: Mobile layout with the hamburger menu

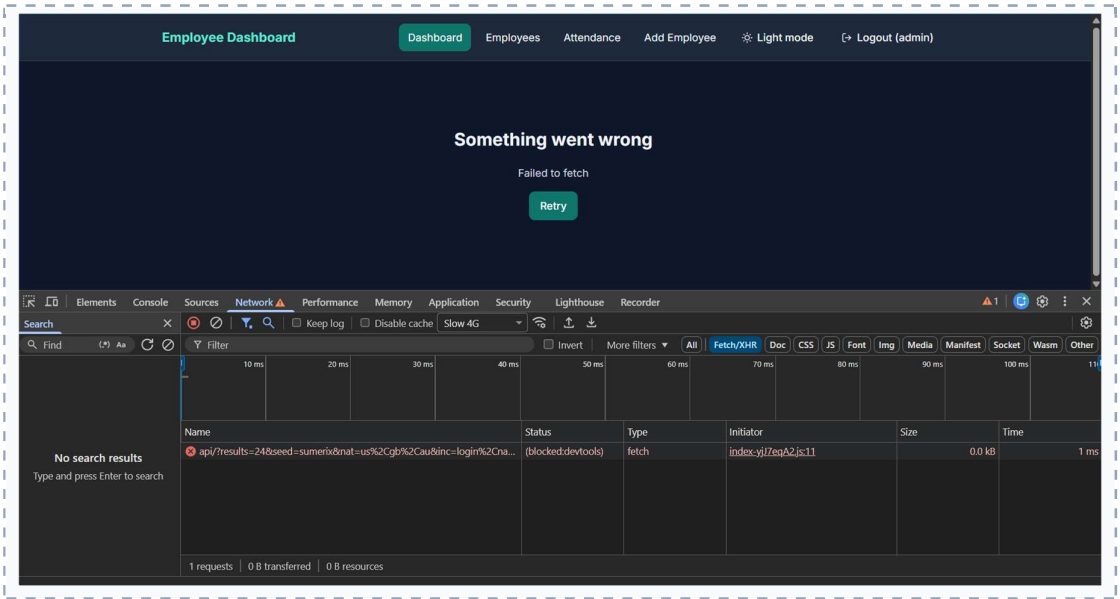


Figure 25: Loading state

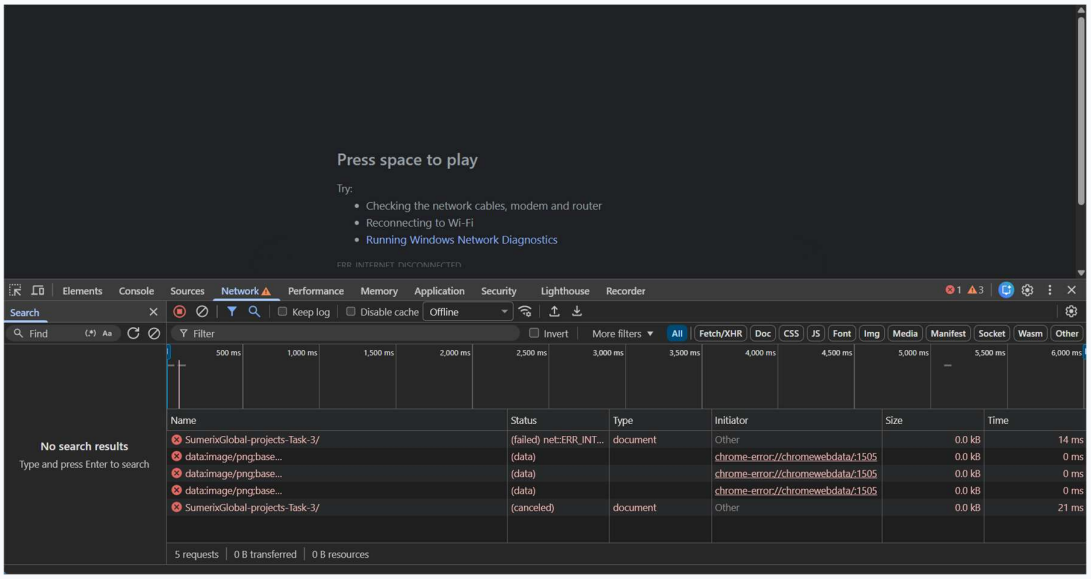


Figure 26: Error state with Retry button

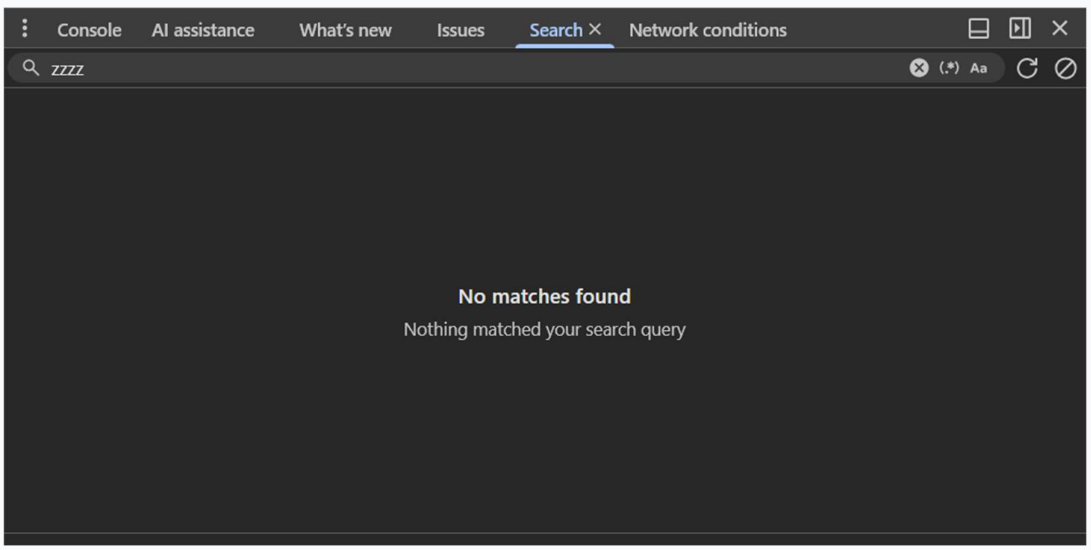


Figure 27: Empty state

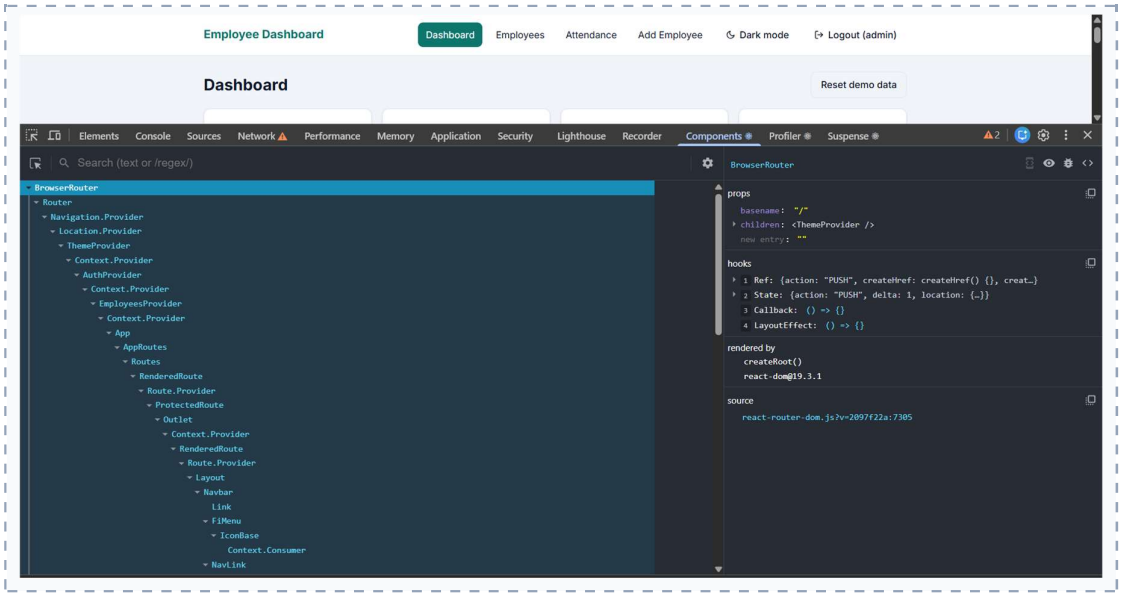


Figure 28: React Developer Tools: component tree, props and state

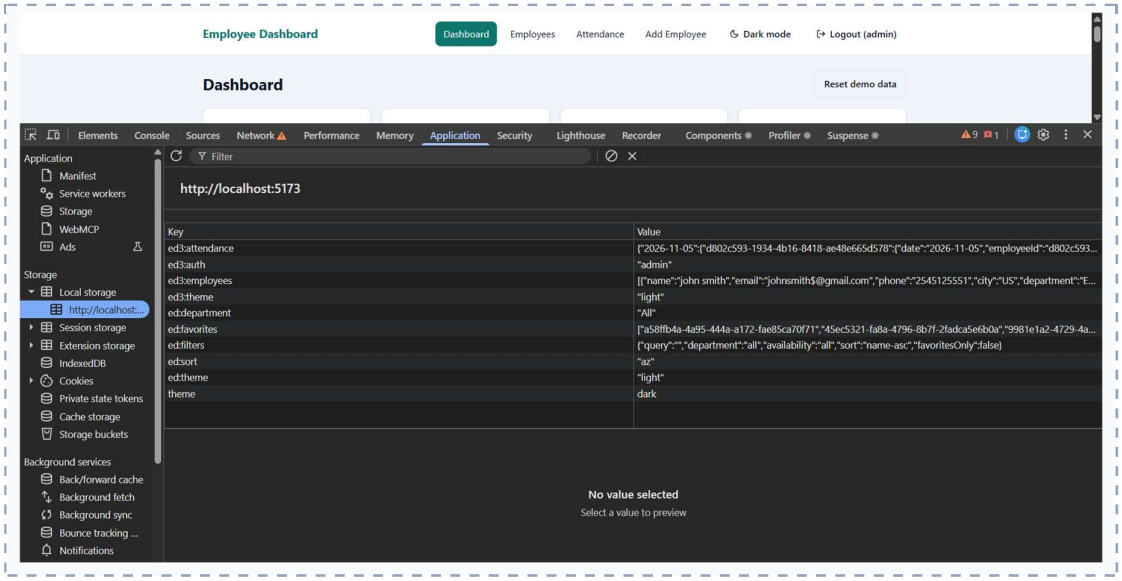


Figure 29: Local Storage keys ed3:employees, ed3:theme, ed3:auth

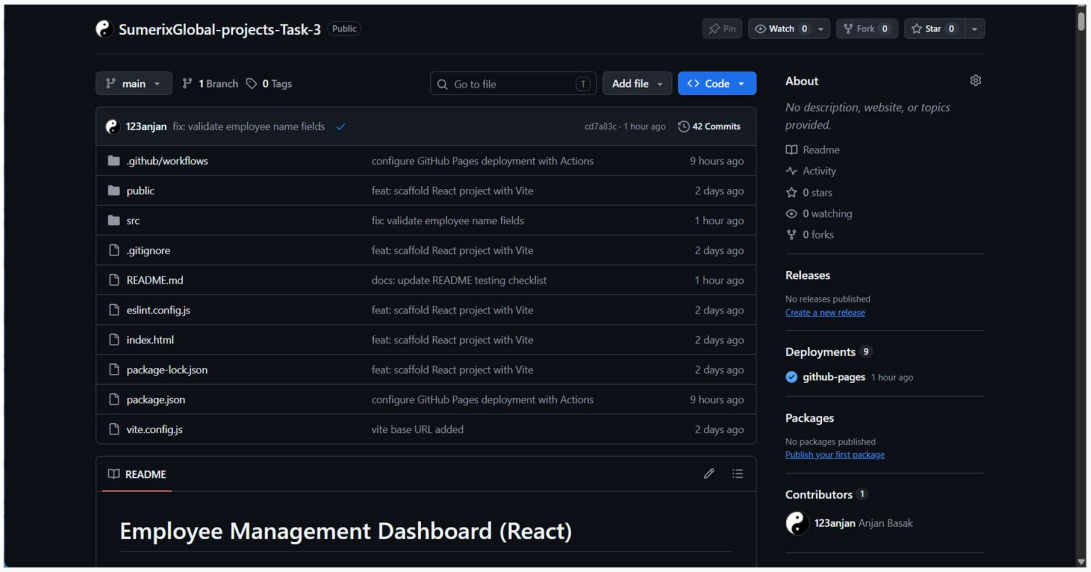


Figure 30: GitHub repository with README

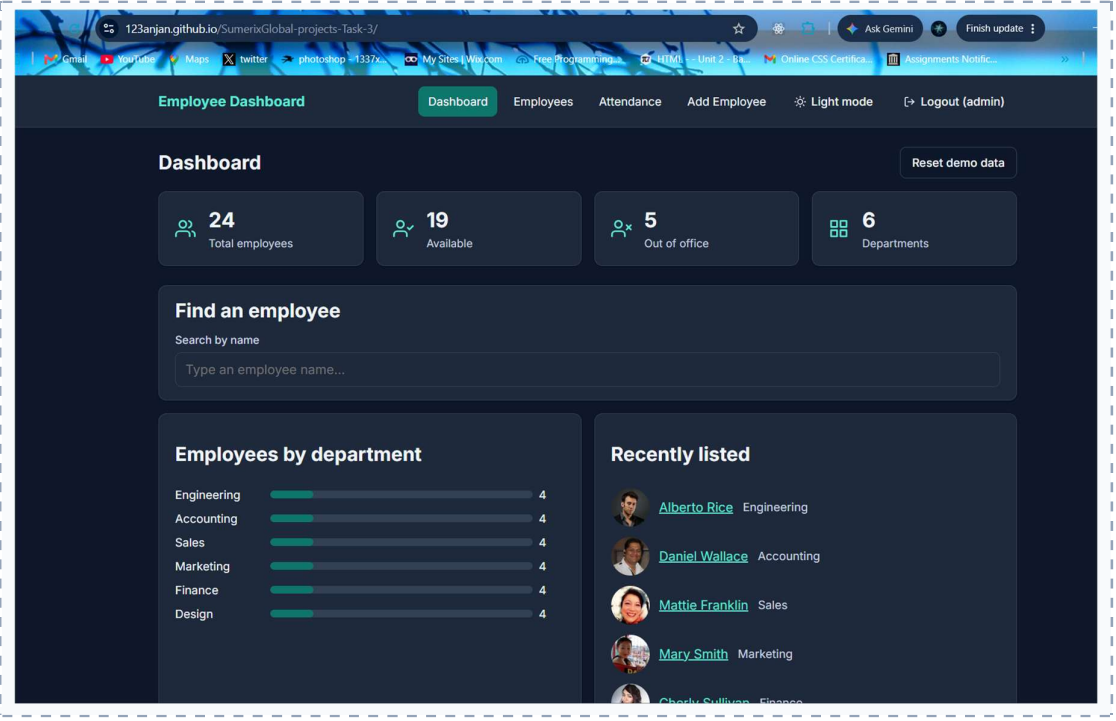


Figure 31: Live deployed application

13. Testing

13.1 Manual test cases

#	Test	Expected result	Result
1	Log in with admin / admin123	Dashboard opens	Pass / Fail
2	Log in with a wrong password	Error message, stays on login	Pass / Fail
3	Open /employees while logged out	Redirected to /login, then returned after login	Pass / Fail
4	Dashboard numbers	Totals match the list; department bars are correct	Pass / Fail
5	Search for part of a name	List filters; clearing restores it	Pass / Fail
6	Department and availability filters, Clear filters	List filters and resets	Pass / Fail
7	Open an employee card	Details page at /employees/	Pass / Fail
8	Add employee with bad data	Validation errors appear; nothing saved	Pass / Fail
9	Add employee with valid data	Appears in the list	Pass / Fail
10	Edit employee	Changes saved and visible	Pass / Fail
11	Delete employee and confirm	Removed from the list	Pass / Fail
12	Refresh the page	Changes and theme are still there	Pass / Fail
13	Toggle dark mode, refresh	Theme remembered	Pass / Fail
14	Open an unknown URL	404 page	Pass / Fail
15	Slow 3G then Reset demo data	Spinner appears	Pass / Fail
16	Offline then Reset demo data	Error and Retry button; Retry works when online	Pass / Fail
17	Search for zzzz	Empty state with Clear filters	Pass / Fail
18	Keyboard only	All actions reachable; focus visible	Pass / Fail
19	Mobile width 375 px	Hamburger menu opens, closes (also with Esc)	Pass / Fail

13.2 Devices and browsers

Environment	How to test	Result
Desktop (1280 px+)	Normal browser window	Pass / Fail
Tablet (about 768 px)	DevTools device toolbar (Ctrl + Shift + M)	Pass / Fail
Mobile (about 375 px)	DevTools device toolbar or a real phone	Pass / Fail
Chrome	Latest version	Pass / Fail
Firefox	Latest version	Pass / Fail
Edge	Latest version	Pass / Fail

When testing on a phone through the local network (http://192.168.x.x:5173), note that `crypto.randomUUID` only exists in secure contexts, so adding an employee can fail there. Test that feature on localhost or on the deployed https site.

13.3 Quality checks

- No red errors in the Console.
- `npm run lint` reports no errors.
- `npm run build` finishes without errors and `npm run preview` works.
- Lighthouse Accessibility score of 90 or higher.
- React Developer Tools shows the expected component tree, props and state.

14. Errors Faced and Fixes

Problem	Cause	Fix
'npm' is not recognized	Node.js missing or PATH not refreshed	Install Node LTS, restart the terminal and VS Code
Failed to resolve import react-router-dom or react-icons	Package not installed	npm install react-router-dom react-icons
Red squiggles in package.json	Missing commas in the scripts block	Add commas between entries; keep valid JSON
react-refresh/only-export-components warning	Context files export a component and a hook	Turn the rule off for src/context/** in eslint.config.js
useTheme must be used inside ThemeProvider	Component rendered outside the provider	Wrap App in all providers in main.jsx
Cannot read properties of undefined (map / filter)	Data not loaded yet	Use default values such as [] and check loading first
Each child in a list should have a key	Missing key in map()	Use a stable id as key, not the index
Page refreshes on form submit	Default form behaviour	Call event.preventDefault() in the handler
Input cannot be typed in	value set without onChange	Use a controlled input with both value and onChange
Infinite re-render loop	State set during render or wrong effect dependencies	Move the update into an effect or handler; fix the dependency array
404 for every URL on dev server	Base path or router basename wrong	Use BASE_URL for the basename; base is '/' in dev
Invalid username or password with empty fields	Expected: credentials do not match	Enter admin / admin123 (optional empty-field check)
Failed to fetch / HTTP 429	No internet or API rate limit	Check connection; wait; use Retry
Dark mode does nothing	Variables not defined for data-theme	Define variables under [data-theme='dark'] and set the attribute
Blank page after deploying	Wrong base path	Set base to '/REPO-NAME/' for build
404 on refresh at /employees	GitHub Pages has no server routing	Copy index.html to 404.html after build
Pages shows raw project	Source branch served instead of the build	Publish dist with gh-pages and select the gh-pages branch
npm run deploy overwrote Task 2 site	Both projects in one repository	Give Task 3 its own repository
Task 3 files appear in Task 2 repo	Task-3 folder inside the Task 2 repo root	Add Task-3/ to the parent .gitignore (no leading spaces) and use a separate repo

Problem	Cause	Fix
fatal: no upstream branch / push rejected	First push needs -u; remote has other commits	git push -u origin main; pull --rebase first if needed
node_modules uploaded	.gitignore missing	Add node_modules and dist, then git rm -r --cached node_modules

14.1 How to debug any problem

- Read the first line of the error message and the file name it mentions.
- Open the browser Console and the terminal; fix the first error first.
- Use React Developer Tools to inspect props and state.
- Add `console.log` to see values, then remove it.
- Search the exact error text, and try a clean reinstall (delete node_modules, run npm install) as a last step.

15. Performance and Best Practices

Technique	Benefit
Data cached in Local Storage	No API call after the first visit; instant start
Debounced search (useDebounce)	List is not re-filtered on every key press
memo on EmployeeCard	Cards re-render only when their props change
useMemo for the filtered list and statistics	Avoids repeating calculations on every render
useCallback for handlers	Gives memoised children a stable function
Keys from stable ids	Efficient list updates
Small single-purpose components	Readable, testable and reusable code
Vite production build	Minified, code-split output in dist/
Semantic HTML and ARIA	Accessibility and SEO-friendly structure

16. Project Plan

Phase	Activities	Duration
1. Learn	React basics, JSX, components, props, state, hooks	Week 1
2. Setup	Vite project, folders, router, global styles, providers	Days 8-9
3. Core UI	Layout, Navbar, login, dashboard, employee cards	Days 10-14
4. Features	Search, filters, details, add, edit, delete, forms, validation	Days 15-20
5. Polish	Dark mode, responsive CSS, accessibility, loading / empty / error states	Days 21-23
6. Test	Manual tests, devices, Lighthouse, lint, build	Days 24-26
7. Document and deploy	README, documentation, GitHub, GitHub Pages	Days 27-28

The durations are a suggested schedule; adjust them to your own timeline.

17. Deployment and GitHub

17.1 Upload to GitHub

```
cd Task-3
git init
git add .
git commit -m "feat: employee management dashboard (React)"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPO-NAME.git
git push -u origin main
```

Commit messages start with `feat:`, `fix:`, `docs:` or `style:` followed by a short description. Task 3 must live in its **own repository**; otherwise deploying would overwrite the Task 2 site.

17.2 GitHub Pages with gh-pages

```
npm run deploy        # builds, then publishes dist/ to the gh-pages branch
# GitHub: Settings > Pages > Source: Deploy from a branch > gh-pages / (root)
```

Before deploying, make sure `base` in vite.config.js equals the repository name. The site address is `https://YOUR-USERNAME.github.io/YOUR-REPO-NAME/`. Vercel and Netlify are alternatives: import the repository, choose the Vite preset, and set the base path to `/`.

18. Connection to Task 2

Task 3 extends Task 2. The concepts and a large part of the logic were reused; the user interface was rebuilt with React.

From Task 2	Used in Task 3
Fetch API and the RandomUser API	services/employeeApi.js calls the same API with the same seed
Mapping raw API data to employee objects	services/employeeService.js with toEmployee()
Safe Local Storage wrapper	services/storage.js is the same code
Debounced search	useDebounce hook, the React version of Task 2's debounce
Loading, error and empty states	Spinner, ErrorMessage and EmptyState components
Dark mode saved in the browser	ThemeContext
Departments and job titles	utils/constants.js
State in, UI out with one render()	React state and automatic re-rendering
Vite and npm	Same tooling, with the React template

18.1 What is not connected

- **Separate folders and repositories:** Task-2 and Task-3 do not import from each other.
- **Separate storage:** Task 3 uses the `ed3:` prefix so the two apps never overwrite each other's data.
- **Different deployments:** each task has its own GitHub Pages site.

18.2 Task 2 versus Task 3

Topic	Task 2 (vanilla JavaScript)	Task 3 (React)
UI updates	Call render() manually	React re-renders when state changes
Structure	ES modules and component functions	JSX components with props
Pages	One screen with a side panel	Many pages with React Router
Global data	A shared state object	Context API providers
Forms	Not required	Controlled forms with validation
Editing data	Read-only directory	Add, edit and delete employees

19. Evaluation Checklist

Requirement	Status
Vite + React project with the recommended folder structure	Done
JSX: expressions, conditionals, lists with keys, fragments	Done
Reusable functional components and props (including children)	Done
useState, useEffect, useRef, useMemo, useCallback	Done
Custom hooks (useLocalStorage, useDebounce, useDocumentTitle)	Done
Controlled forms with validation and error messages	Done
React Router: nested, dynamic and protected routes, 404	Done
Context API: theme, authentication, employees	Done
CSS Modules, global CSS, responsive layout, icons, Google Fonts	Done
Loading, empty and error states	Done
Dashboard, list, search, filter, add, edit, delete	Done
Dark mode and responsive navigation	Done
Accessible UI (labels, ARIA, keyboard, focus)	Done
README, documentation, screenshots and GitHub repository	Complete the screenshots
Deployed site (optional)	Add the live link

20. Challenges and Lessons Learned

Challenge	Lesson
Understanding when a component re-renders	State, props and context changes trigger renders; use React DevTools to watch them
Passing data through many levels	Context removes prop drilling for global data
Effect dependency arrays and infinite loops	List every dependency and keep effects small
Refreshing nested URLs on GitHub Pages	Base path, router basename and the 404.html copy
Keeping Task 2 and Task 3 apart	Separate repositories and storage key prefixes
Linting context files	Understand Fast Refresh rules and configure ESLint deliberately
Testing on real devices	Secure-context limits (crypto.randomUUID) differ between localhost and the local network

21. Future Enhancements and Task 4

21.1 Future enhancements

- Real authentication with a backend, tokens and role-based access.
- Database and REST API for employees instead of Local Storage.
- Pagination or infinite scrolling for large lists.
- Sorting options, bulk actions and export to CSV.
- Image upload for employee photos.
- Automated tests with Vitest and React Testing Library.
- Internationalisation and more accessibility checks.

21.2 Connection to Task 4

Task 4 covers advanced React, custom hooks, state management, Redux Toolkit, API integration, authentication, protected routes, React Query, form validation, UI component libraries and enterprise front-end architecture. This project prepares for it: contexts can be replaced or complemented by Redux Toolkit, the service layer is ready for a real API, the demo login can become token-based authentication, and the protected-route structure is already in place.

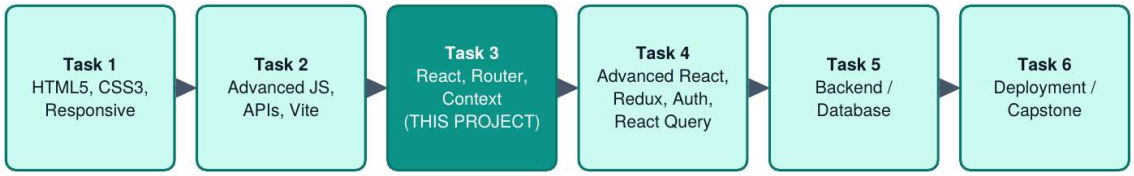


Figure 33: Internship roadmap and the position of this project

22. Conclusion

The Employee Management Dashboard shows how a React single-page application is built from the ground up: a clear folder structure, reusable components, props and state, hooks, validated forms, protected routing, global state with Context, persistent data and an accessible responsive design. It reuses the API, service and storage ideas of Task 2, and it prepares the foundation for the advanced React and back-end topics of the following tasks.

23. References

- React documentation: https://react.dev
- React Router documentation: https://reactrouter.com
- Vite documentation: https://vite.dev
- RandomUser API: https://randomuser.me
- MDN Web Docs: https://developer.mozilla.org
- gh-pages package: https://www.npmjs.com/package/gh-pages
- WAI-ARIA Authoring Practices: https://www.w3.org/WAI/ARIA/apg/